22

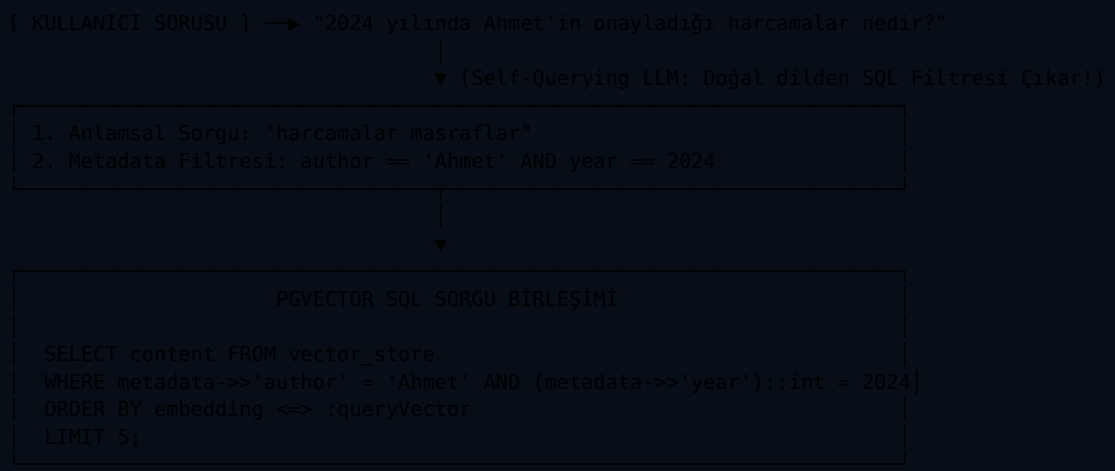
BÖLÜM: ÜRETIM SEVIYESINDE RAG: TEMELLER, CHUNKING & ETL | MODULE: PRODUCTION RAG CORE: FUNDAMENTALS, CHUNKING & INGESTION PIPELINES

Metadata Filtreleme: Self-Querying & Hibrit Vector + SQL Filtreleme

Metadata Filtering: Self-Querying Retrievers & Hybrid Vector + SQL Predicates

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanede 'Bana 2024 yılında basılmış, konusu aşk olan ve mavi kapaklı kitapları getir' dediğinizi düşünün. Eğer kütüphaneci sadece 'aşk' kelimesine göre ararsa önünüze 1950'den kalma 500 tane kitap yığar. Akıllı bir kütüphaneci önce filtre koyar: 'Sadece 2024 basımı ve mavi kapaklıları ayır' (SQL Filtresi); sonra kalan 5 kitap arasından en güzel aşk romanını seçer (Vektör Araması)! Self-Querying tam olarak budur: Kullanıcının cümlesindeki '2024' ve 'Ahmet' kelimelerini anlayıp otomatik veritabanı filtresine dönüştürür!

🇬🇧 ENGLISH (ELI5)

If you ask a librarian: 'Find me romance novels published in 2024 by French authors', searching purely on the word 'romance' drowns you in 10,000 irrelevant books from the 1800s. An intelligent librarian filters first: 'Publication Year = 2024 AND Nationality = French' (Relational SQL Predicate), and then runs semantic similarity strictly on the filtered pool (Vector Search). Self-Querying automatically extracts structured metadata predicates from natural language queries!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Filtreleme Mimarisi Türleri: 1) Pre-Filtering (Arama Öncesi): Önce SQL WHERE koşulu ile alakasız satırlar elenir, kalan alt kümede vektör araması yapılır. En güvenli ve hızlı yöntemdir. 2) Post-Filtering (Arama Sonrası): Önce en yakın 100 vektör çekilir, sonra bellekte filtre uygulanır. Tehlikelidir: Eğer ilk 100 vektörün içinde 'Ahmet'in 2024 dokümanı yoksa sonuç bomboş döner! 3) Self-Querying Retriever: Küçük ve hızlı bir model (GPT-4o-mini), kullanıcının serbest metnini ayrıştırarak Spring AI `FilterExpression` nesnesi üretir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Filtering Mechanics & Architecture: 1) Pre-Filtering: Applies metadata relational predicates (`WHERE tenant_id = 'org_1'`) inside the database engine prior to computing HNSW graph traversals. Guarantees security and bounded search spaces. 2) Post-Filtering: Fetches top-K semantic neighbors and filters in-memory on application servers. Highly vulnerable to recall collapse: if the initial K results lack target metadata matches, the system returns empty sets! 3) Self-Querying: Uses an auxiliary LLM step to translate natural language into structured AST filter expressions.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: Metadata Filtresi ile Güvenli Çok Kiracılı (Multi-Tenant) Arama
import org.springframework.ai.document.Document;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.ai.vectorstore.SearchRequest;
import java.util.List;

public class SecureTenantSearchService {
    private final VectorStore vectorStore;

    public SecureTenantSearchService(VectorStore vectorStore) {
        this.vectorStore = vectorStore;
    }

    public List<Document> searchTenantData(String userQuery, String tenantId, int minYear) {
        // Pre-filtering: Sadece kullanıcının şirketine ait ve belirtilen yıldan büyük kayıtlar
taranır!
        String filterExpression = String.format("tenantId == '%s' && year >= %d", tenantId, minYear);

        return vectorStore.similaritySearch(
            SearchRequest.builder()
                .query(userQuery)
                .topK(5)
                .similarityThreshold(0.75)
                .filterExpression(filterExpression)
                .build()
        );
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Güvenlik Açığı (Tenant Isolation Leak): Çok kiracılı (Multi-Tenant) kurumsal sistemlerde metadata filtresinde `tenantId` zorunlu tutulmazsa, A şirketinin kullanıcısı B şirketinin gizli maaş veya finans dokümanlarını vektör aramasıyla bulup okuyabilir!

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Critical multi-tenant vulnerability: Never rely on soft prompt instructions to enforce data tenancy boundaries. Tenant ID filters must be hardcoded at the database query layer (`tenantId == X`) to prevent cross-customer data leakage.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Pre-filtering, veri sızıntılarını ve arama alanı kirlenmesini önleyen en güvenli yöntemdir.

• Self-Querying, doğal dildeki tarih, yazar, kategori gibi kriterleri otomatik SQL filtrelerine dönüştürür.

• Çok kiracılı (SaaS) sistemlerde tenantId filtresi uygulama kodunda zorunlu kılınmalıdır.

🎯 Key Takeaways for the Interview

• Pre-filtering is strictly superior to post-filtering for enterprise security and recall.

• Self-Querying converts colloquial constraints into structured database WHERE predicates.

• Tenant ID predicates must be enforced non-negotiably in SaaS vector architectures.